

React **useEffect** Hooks

Hook `useEffect` 允許您在組件中執行屬性更改關聯作用。

屬性更改關聯作用的範例有：獲取數據、直接更新 **DOM** 和計時器。

`useEffect` 接受兩個參數。第二個參數是可選擇性使用。

`useEffect(<function>, <dependency>)`

讓我們以定時器為例。

用於 `setTimeout()` 在初始計算 1 秒後渲染：

MyTimer.js

```
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function MyTimer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });

  return <h1>I have rendered {count} times!</h1>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyTimer />);

export default MyTimer;
```

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import MyTimer from './MyTimer';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <MyTimer />
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();
```

可是等等！它一直在計數，即使它應該只計數一次！

`useEffect` 在每個渲染器上運行。這意味著當計數發生變化時，會發生渲染，然後觸發另一個 `Effect`。

這不是我們想要的。有幾種方法可以控制 `Effect` 何時運行。

我們應該包含接受陣列的第二個參數。我們可以選擇將依賴項傳遞給 `useEffect` 陣列也就是說第二個參數可以決定是否再執行 `Effect`。

例子

1.沒有第二個依賴參數：

```
useEffect(() => {
  //Runs on every render
});
```

例子

2. 一個空陣列：

```
useEffect(() => {  
  //只有執行 1 次  
}, []);
```

```
import { useState, useEffect } from "react";  
import ReactDOM from "react-dom/client";
```

```
function MyTimer2() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    setTimeout(() => {  
      setCount((count) => count + 1);  
    }, 1000);  
  }, []); // <- add empty brackets here  
  
  return <h1>I've rendered {count} times!</h1>;  
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<MyTimer2 />);
```

```
export default MyTime2
```

useEffect 下面是一個依賴於變數的 **Hook** 範例。如果 **count** 變數更新，**Effect** 將再次運行：

```
import { useState, useEffect } from "react";  
import ReactDOM from "react-dom/client";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  const [calculation, setCalculation] = useState(0);  
  
  useEffect(() => {
```

```

    setCalculation(() => count * 2);
  }, [count]); // <- add the count variable here

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      <p>Calculation: {calculation}</p>
    </>
  );
}

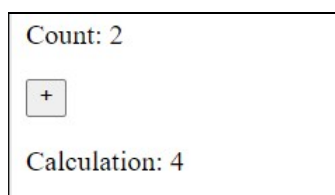
```

```

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Counter />);

```

export default Counter;



EmployeeComponent.js

```
import React, {useState, useEffect} from 'react'
```

```
function EmployeeComponent() {
```

```
  const [employees, setEmployees] = useState([])
```

```
  useEffect(() => {
    getEmployees()
  }, [])
```

```
  const getEmployees = () => {
```

```
    setEmployees([
      {
```

```

        "id":1,
        "firstName":"Ramesh",
        "lastName":"Fadatare",
        "email":"ramesh@gmail.com"
    },
    {
        "id":2,
        "firstName":"Tony",
        "lastName":"Stark",
        "email":"tony@gmail.com"
    },
    {
        "id":3,
        "firstName":"John",
        "lastName":"Cena",
        "email":"cena@gmail.com"
    }
    ])
};

return (
    <div className = "container">

        <h1 className = "text-center"> Employees List</h1>

        <table className = "table table-striped">
            <thead>
                <tr>
                    <th> Employee Id</th>
                    <th> Employee First Name</th>
                    <th> Employee Last</th>
                    <th> Employee Email</th>
                </tr>

            </thead>
            <tbody>
                {
                    employees.map(

```

```

        employee =>
        <tr key = {employee.id}>
            <td> {employee.id }</td>
            <td> {employee.firstName }</td>
            <td> {employee.lastName }</td>
            <td> {employee.email }</td>

        </tr>

    )
    }
</tbody>
</table>

</div>
)
}

```

export default EmployeeComponent

npm install bootstrap 安裝 bootstrap

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import EmployeeComponent from './Employee';
import reportWebVitals from './reportWebVitals';
import 'bootstrap/dist/css/bootstrap.min.css';

```

```

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <EmployeeComponent/>
  </React.StrictMode>
);

```

```

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals

```

reportWebVitals();

Employees List			
Employee Id	Employee First Name	Employee Last	Employee Email
1	Ramesh	Fadatare	ramesh@gmail.com
2	Tony	Stark	tony@gmail.com
3	John	Cena	cena@gmail.com